Faculty of Engineering and Technology
Object Oriented Programming (CEUE203)
A.Y 2026 – 27 Semester – 3
Practical List

Practical Number		CO/PO
1	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. **1. Vending machine (VendingMachine.java, main in this file)** (a) Define an enum Coin with constants ONE, TWO, FIVE, TEN. (b) In main, set a snack price of 15 and a running total of 0; create a Scanner. (c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far. (d) Stop the loop once the total reaches 15 or more. (e) Print the change to return (total − 15). **Expected output:** *TEN then FIVE prints "Paid. Change: 0"; TEN then TEN prints "Paid. Change: 5".* **2. Toll booth (TollBooth.java, main in this file)** (a) Define a record Vehicle(String number, String type). (b) In main, keep a running toll total and three counters (bike, car, truck). (c) Loop until the user types "done" for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type's counter. (d) After the loop, print the total toll and the type with the highest count. **Expected output:** *two cars + one bike → "Total toll: 120", "Most frequent: car".* **3. (For Advanced Learners – home) Rock-Paper-Scissors-Lizard-Spock (RPSLS.java)** (a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK. (b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases). (c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call winner(), print the round result and keep score. (d) After 5 rounds, print the overall winner. **Expected output:** *each round prints both moves and the round winner; ends with a "You win 3–2" summary.* **Part B — Project Work (Hour 2): MiniBank** MiniBank is the console banking application you will build across the whole semester. In this first lab you set up your tools and create the project and its menu shell — the screen the user navigates. Every later lab attaches one real banking feature behind these menu options. 1. Set up your environment: open the project in GitHub Codespaces (or install JDK 17+ and an IDE locally), and create a private GitHub repository named oop-minibank-<rollno>; add the faculty as a collaborator. 2. Create a public class named MiniBank that contains the main method. 3. Define a record named BankInfo with two String fields, name and branch (a record is a short, read-only class; Java auto-generates its constructor and field accessors). Print one BankInfo object as the application header. 4. Define an enum named MenuOption with the constants OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER and EXIT — these are the actions MiniBank will offer. 5. Display the numbered menu to the user and read the chosen number with a Scanner. 6. Use a switch expression on the choice to print a short placeholder for that option (for example, "Deposit — to be implemented in a later lab"). Keep showing the menu in a loop until the user selects EXIT, then print a goodbye message. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is the role of the JVM, and what file does the javac compiler produce?	**CO1** PO1, PO2, PO3

Practical Number		CO/PO
	2. How does a switch expression differ from a traditional switch statement? 3. Why is an enum a good choice for a fixed set of menu options, and a record for read-only data? **Supplementary Problems –** • Re-prompt the user when an invalid menu number is entered. • Add a menu option that prints the bank's working hours. **Key Skills to be addressed –** Java program structure; compiling and running with javac/java; enum and record; switch expression; reading input with Scanner; basic git (commit, tag). **Applications –** The starting point of every menu-driven console application — ATM front-ends, point-of-sale terminals, kiosk software. **Learning Outcome –** The student can set up the Java toolchain and write, compile, run and version-control a basic interactive program. (CO1)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. The user types menu choices (e.g., 1, 2, 3, 4) at runtime.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-01. Write a short README describing how to compile and run. Verify the menu loops correctly for every option, including EXIT.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Program compiles and menu runs – 40%; correct use of enum, record and switch expression – 30%; code clarity and naming conventions – 15%; viva (JVM, switch expression, record) – 15%.	
	Expected Input / Process / Output **Input:** The user enters a menu number, for example 2 (Deposit). **Process:** main reads the number with Scanner; the switch expression maps it to the matching MenuOption and prints the placeholder; the loop then redraws the menu. **Expected Output:** The header and menu are shown; entering 2 prints "Deposit — to be implemented in a later lab"; entering the EXIT number prints a goodbye message and stops the program.	

Practical Number		CO/PO
2	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. **1. Smart thermostat (Thermostat.java, main in this file)** (a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount. (c) Constructor(location): chain with this(location, 22). (d) raise(): if temperature < MAX add 1, else print "Already at maximum (30)". (e) lower(): if temperature > MIN subtract 1, else print "Already at minimum (16)". (f) Add getTemperature() (no setter) and static getActiveCount(). (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount().	**CO1** PO1, PO2, PO3

Practical Number		CO/PO
	Expected output: *temperature rises to 30 then "Already at maximum", falls to 16 then "Already at minimum", count 2.* **2. Cinema show (CinemaShow.java, main in this file)** (a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0. (b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100). (c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged. (d) cancel(int n): increase seatsAvailable by n but never above capacity. (e) Add getSeatsAvailable() and static getTotalBooked(). (f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked(). **Expected output:** *booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.* **3. (For Advanced Learners – home) Parking lot (ParkingLot.java)** (a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0. (b) park(String type): for "two", if twoWheelers < twoCap increment and add 20 to revenue; for "four", if fourWheelers < fourCap increment and add 40; otherwise print "Full" and reject. (c) leave(String type): decrement the matching count but never below 0. (d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue. **Expected output:** *parking into a full section is refused; revenue = 20×(two parked) + 40×(four parked).* **Part B — Project Work (Hour 2): MiniBank** In Practical 1 you built the empty menu shell. Now you create the data the bank works on — the Customer and Account classes. Using encapsulation, an account's balance can change only through deposit() and withdraw(), never directly. These two classes are the foundation every later lab extends. 1. Create a class named Customer with private fields name, email and mobile (all String) and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as "CUST101". Add a constructor and public getter methods. 2. Create a class named Account with private fields: a final String accountNumber, a String ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example "AC0001"). 3. Add two constructors to Account: one taking ownerName and an opening balance; one taking only ownerName that calls the first using this(ownerName, 0). 4. Add a public method deposit(long amount) that adds amount to balance. Add a public method withdraw(long amount) that subtracts amount and returns true when the balance is sufficient, otherwise returns false without changing the balance. Add getter methods. Do NOT add a public setter for balance. 5. In main, create three Account objects inside an Account[] array, perform a few deposits and withdrawals, and print each balance. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is encapsulation, and how do private fields with public getters enforce it? 2. Why is accountNumber declared final? 3. What is the difference between a static field and an instance field when generating ids? **Supplementary Problems –** • Add a helper that moves an amount from one Account to another using withdraw and deposit. • Reject a deposit or withdrawal of a negative amount (return without changing the balance).	

Practical Number		CO/PO
	Key Skills to be addressed – Defining classes and objects; instance vs static members; constructors and constructor chaining with this(); access modifiers; encapsulation; arrays of objects. **Applications –** Modelling real-world entities (accounts, customers, products) with controlled, valid state is the foundation of every business application. **Learning Outcome –** The student can design well-encapsulated classes and create and manipulate objects safely. (CO1)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. Sample objects are created in code (for example owner "Riya" with opening balance 5000).	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-02. Confirm that balance cannot be modified from outside the class and that withdraw returns false when funds are insufficient.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct classes and encapsulation – 40%; constructors and static id generation – 25%; deposit/withdraw logic – 20%; naming conventions and viva – 15%.	
	Expected Input / Process / Output **Input:** In code: create an account with opening balance 5000, then call deposit(2000), withdraw(3000) and withdraw(10000). **Process:** deposit adds to the balance; the first withdraw succeeds because the balance is sufficient and reduces it; the second withdraw fails and leaves the balance unchanged. **Expected Output:** Balance prints as 7000 after the deposit, 4000 after the successful withdraw, and remains 4000 after the failed withdraw (which returns false).	

Practical Number		CO/PO
3	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. **1. Distinct points (Point.java + Driver.java with main)** (a) Create class Point with private int x, y and a constructor. (b) Override toString() to return "(x, y)". (c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)). (d) In Driver.main: build a Point[] with some repeated coordinates. (e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not. (f) Print "Distinct: N". **Expected output:** *5 points with 2 repeats → "Distinct: 3".* **2. Duplicate card check (Card.java + Driver.java)** (a) Create class Card with private String rank, suit and a constructor. (b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts"). (c) Override equals()/hashCode() by rank + suit. (d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards. (e) Print "Duplicate found: <card>" the first time a repeat appears. **Expected output:** *a second "Ace of Spades" → "Duplicate found: Ace of Spades".*	**CO1** PO1, PO2, PO3

Practical Number		CO/PO
	3. (For Advanced Learners – home) Fractions (Fraction.java + Driver.java) (a) Create class Fraction with private int num, den. (b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g. (c) Override toString() to return num + "/" + den. (d) Override equals()/hashCode() using the reduced num and den. (e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same. **Expected output:** *new Fraction(2,4) prints "1/2" and equals new Fraction(1,2).* **Part B — Project Work (Hour 2): MiniBank** In Practical 2 you created the Account and Customer classes. Here you make their objects behave correctly — they print as readable text (toString) and are compared by account number (equals/hashCode) — and you add a nested Address class. Correct equals/hashCode is essential because Practical 12 will store accounts in collections that rely on it. 1. In Account, override the toString() method to return a readable line containing accountNumber, ownerName and balance. 2. In Account, override equals(Object o) and hashCode() so that two accounts are equal when their accountNumber values are equal. 3. In Customer, add a public static nested class named Address with String fields line, city and pincode plus getters; add an Address field to Customer and a getAddress() method. 4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the customer. 5. In main, print accounts using toString(), compare two Account objects with equals(), and use instanceof to check an object's type. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. Why must equals() and hashCode() be overridden together? 2. What does the default Object.toString() return, and why override it? 3. What is the difference between a static nested class and an inner class? **Supplementary Problems –** • Make toString() also show whether the account is active. • Add a method that returns a formatted, multi-line statement string for an account. **Key Skills to be addressed –** Overriding Object-class methods (toString, equals, hashCode, clone); the equals/hashCode contract; static nested classes; the instanceof operator. **Applications –** Readable, comparable objects are needed everywhere — logging, searching, and (later in this project) storing objects in collections. **Learning Outcome –** The student can make objects behave correctly for printing and comparison and can use nested classes. (CO1)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. Two Account objects with the same accountNumber are created to test equality.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-03. Verify that two accounts with the same number are reported equal and that the toString() output is readable. (The @Override annotation is introduced later, in Practical 7.)	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct toString/equals/hashCode – 40%; nested class and clone – 25%; instanceof use – 15%; naming conventions and viva – 20%.	
	Expected Input / Process / Output	

Practical Number		CO/PO
	Input: In code: two Account objects a1 and a2 created with the same accountNumber; one Customer object that has an Address. **Process:** equals() compares the accountNumber values; toString() builds the display string; instanceof checks the object's type. **Expected Output:** Printing a1 shows a readable line such as "AC0001 | Riya | 4000"; a1.equals(a2) prints true; the instanceof check prints true for an Account.	

Practical Number		CO/PO
4	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. **1. Password strength (PasswordChecker.java + Driver.java)** (a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods. (b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0–1), "Medium" (2–3) or "Strong" (4). (c) In Driver.main: test several passwords; for each, print which rules passed and the final label. **Expected output:** *"abc" → Weak; "Abcd1234!" → Strong.* **2. Chat-log filter (ChatFilter.java + Driver.java)** (a) In Driver.main, define a String[] of log lines like "10:05 alice Hello there" (include one malformed line with fewer than 3 parts). (b) Read a keyword from the user. (c) For each line, split into time/user/message with split(" ", 3); skip lines that do not have 3 parts. (d) If the message contains the keyword (case-insensitive), increment a counter and append "time user: message" to a StringBuilder. (e) Print "Matches: N" and the built report. **Expected output:** *keyword "hello" over 3 lines (1 malformed) → "Matches: 1" and the matching line.* **3. (For Advanced Learners – home) Template filler (TemplateFiller.java + Driver.java)** (a) In Driver.main, define a template like "Dear {name}, order {id} ships {date}." and two parallel arrays: names {"name", "id"} and values {"Riya", "A07"} (note {date} is not supplied). (b) Compile a Pattern for \{(\w+)\} and get a Matcher on the template. (c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with "[?]" if not found. (d) Print the filled template. **Expected output:** *the unsupplied {date} becomes [?].* **Part B — Project Work (Hour 2): MiniBank** Your classes exist, but the bank must never trust raw user input. In this lab you build MiniBank's validation and command-parsing layer — regex checks for mobile, email, PAN and IFSC, and a parser that turns a typed line such as "DEPOSIT AC0001 500" into a Command object. Practical 14 will run every menu action through this layer. 1. Create a class named Validator with public static methods that use compiled regular expressions (Pattern): isValidMobile(String), isValidEmail(String), isValidPan(String) and isValidIfsc(String); each returns a boolean. 2. Define an enum named TransactionType with the constants DEPOSIT, WITHDRAW and TRANSFER. Then define a record named Command with three fields: a TransactionType type, a String accountNumber and a long amount.	**CO1** PO1, PO2, PO3

Practical Number		CO/PO
	3. Create a class named CommandParser with a public static method parse(String line) that splits a line such as "DEPOSIT AC0001 500" with split(), converts the parts, and returns a Command object. 4. Create a class named StatementFormatter with a public static method buildStatement(Account account) that uses a StringBuilder to assemble a multi-line account statement string. 5. In main, test each validator with one correct and one wrong input, then parse a sample command and print its three parts. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is the difference between String, StringBuilder and StringBuffer, and when is each preferred? 2. What does the regular expression [6-9][0-9]{9} match, and why use anchors? 3. Why should input be validated at the boundary before it is used? **Supplementary Problems –** • Add isValidAmount(String) that accepts only positive whole numbers. • Make parse() report an error when the line has the wrong number of parts. **Key Skills to be addressed –** String methods; StringBuilder; regular expressions with Pattern and Matcher; tokenising with split(); records. **Applications –** Form validation, command interpreters and data-cleaning routines in every real application. **Learning Outcome –** The student can validate and parse text input reliably using strings and regular expressions. (CO1)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. Test strings are supplied in code, e.g. "9876543210", "abc@xyz.com", "DEPOSIT AC0001 500".	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-04. Confirm each validator returns true for valid input and false for invalid input, and that parse() returns the correct Command.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct regex validators – 35%; Command record and parser – 30%; StringBuilder statement – 15%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: validate "9876543210" and "12345"; parse the line "DEPOSIT AC0001 500". **Process:** Each validator matches the input against its Pattern; parse() splits the line into three tokens and builds a Command. **Expected Output:** isValidMobile("9876543210") prints true and isValidMobile("12345") prints false; the parsed command prints type=DEPOSIT, accountNumber=AC0001, amount=500.	

Practical Number		CO/PO
5	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home.	**CO1, CO2**

Practical Number		CO/PO
	1. Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle, Triangle. Put a mix in a Shape[] and loop once to print each area and the running total, tracking the largest — one loop handling every shape through polymorphism. 2. Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns. 3. (For Advanced Learners – home) Model a few media types that each compute a late fee differently and total the fees for a returned batch. Design the hierarchy and rules yourself. **Part B — Project Work (Hour 2): MiniBank** A real bank has different account types. Building on the single Account class, this lab makes Account abstract and adds three subclasses — SavingsAccount, CurrentAccount and FixedDepositAccount — each with its own interest and withdrawal rules chosen automatically by polymorphism. This lets the rest of MiniBank treat every account through one common type. 1. Make Account an abstract class. Add two abstract methods: double interestRate() and boolean canWithdraw(long amount). 2. Create a class SavingsAccount that extends Account with a long field minBalance; interestRate() returns 4.0; canWithdraw() returns true only if the balance stays greater than or equal to minBalance. 3. Create a class CurrentAccount that extends Account with a long field overdraftLimit; interestRate() returns 0; canWithdraw() allows the balance to fall to minus overdraftLimit. 4. Create a class FixedDepositAccount that extends Account; interestRate() returns 7.0; canWithdraw() returns false (the deposit is locked). 5. Call super(...) from each subclass constructor to initialise the inherited fields. 6. In main, place objects of the three types in an Account[] array, loop through it and call interestRate() on each (this demonstrates run-time polymorphism); use a pattern instanceof check to handle one type specially. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is the difference between an abstract class and a concrete class? 2. What is dynamic method dispatch (run-time polymorphism)? 3. What does the super keyword do in a subclass constructor? **Supplementary Problems –** • Add a method monthlyInterest() in Account that uses interestRate() and balance. • Override canWithdraw() in FixedDepositAccount to allow withdrawal after a maturity date. **Key Skills to be addressed –** Inheritance with extends; abstract classes and methods; method overriding; run-time polymorphism; super; object casting; pattern matching with instanceof. **Applications –** Type hierarchies (account types, employee types, shape types) are used wherever many variants share common behaviour. **Learning Outcome –** The student can design a class hierarchy and use polymorphism to write extensible code. (CO1, CO2)	PO1, PO2, PO3
	Dataset / Test Data (Source and Description, if applicable): No external dataset. One object of each account type is created in code.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-05. Confirm that calling interestRate() on the array prints the correct rate for each subtype.	

Practical Number		CO/PO
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct abstract base and subclasses – 40%; polymorphic behaviour – 25%; super and instanceof – 15%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: a SavingsAccount, a CurrentAccount and a FixedDepositAccount placed in an Account[] array. **Process:** The loop calls interestRate() on each element; Java selects the subclass version at run time (polymorphism). **Expected Output:** The program prints 4.0 for the savings account, 0.0 for the current account and 7.0 for the fixed deposit.	

Practical Number		CO/PO
6	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda. 2. Notification senders: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent. 3. (For Advanced Learners – home) Build a small discount engine where each rule is a lambda the user picks at runtime and applies to a list of prices; organise it into packages and ship it as a runnable JAR. **Part B — Project Work (Hour 2): MiniBank** With several classes now in place, this lab gives MiniBank a clean structure. You express account capabilities as interfaces, organise the code into model, util and service packages, and build a runnable JAR — the form in which the finished application will actually be delivered and run. 1. Create an interface named Transactable with methods void deposit(long amount) and boolean withdraw(long amount); make Account implement it. 2. Create an interface named InterestBearing with a default method double yearlyInterest() that uses interestRate() and the balance. 3. Create a functional interface named WithdrawRule with a single method boolean allow(Account account, long amount). Use it two ways in main: first with an anonymous class, then with a lambda expression. 4. Create a marker interface named Premium (an interface with no methods). 5. Move all classes into the packages model, util and service (add package statements and fix the import statements). Use one static import. 6. Build a runnable JAR file with a manifest whose Main-Class is the MiniBank class, and run it with the command java -jar. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is the difference between an interface and an abstract class? 2. What is a functional interface, and how does a lambda use it? 3. Why is a default method useful in an interface? **Supplementary Problems –** • Add a second default method to InterestBearing and call it from main. • Add a static import for the Validator methods and use them without the class name. **Key Skills to be addressed –** Interfaces; default, static and private interface methods; functional interfaces with lambdas; anonymous classes; marker interfaces; packages; static import; building a JAR.	**CO2** PO1, PO3, PSO2

Practical Number		CO/PO
	Applications – Interface-based design and packaging are the basis of every maintainable, modular codebase and of plug-in architectures. **Learning Outcome –** The student can design interface-based code, organise it into packages, and produce a runnable build artifact. (CO2)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-06. Confirm the project still compiles after moving to packages and that java -jar runs the program.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct interfaces (incl. functional and marker) – 35%; package organisation and static import – 25%; runnable JAR – 20%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: a WithdrawRule implemented first as an anonymous class, then as a lambda; the built JAR run from the command line. **Process:** The anonymous class and the lambda both provide the allow() logic; the JAR manifest tells java which class has main. **Expected Output:** Both versions return the same allow() result for the same inputs; java -jar minibank.jar launches the menu from Practical 1.	

Practical Number		CO/PO
7	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object. 2. Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods. 3. (For Advanced Learners – home) Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column. **Part B — Project Work (Hour 2): MiniBank** Now that you are comfortable with classes and interfaces, you add metadata to MiniBank. You define custom annotations (@Id, @Positive, @MaxLength) and read them at run time with reflection to validate any object automatically — the same idea real frameworks use. The @Id marker will later identify each record in the generic repository you build in Practical 13. 1. Add the @Override annotation above every overridden method written so far (toString, equals, hashCode and the subclass methods). Add @FunctionalInterface above the WithdrawRule interface. 2. In a package named model.annotation, define three custom annotations: @Id (a marker, with @Target(FIELD) and @Retention(RUNTIME)); @Positive with a member String message() default "must be > 0"; and @MaxLength with a member int value(). 3. Put @Id on the accountNumber field and @Positive on the balance field of Account.	**CO1, CO2** PO2, PO3, PSO2

Practical Number		CO/PO
	4. Create a class named AnnotationValidator with a public static method String[] validate(Object obj) that uses reflection — getClass().getDeclaredFields(), field.isAnnotationPresent(...), field.setAccessible(true) and field.get(obj) — to check the @Positive and @MaxLength rules and return the error messages as a String[]. 5. In main, create an Account with a negative balance and run AnnotationValidator.validate() to print the error. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What are the meta-annotations @Retention and @Target used for? 2. What is reflection, and what does getDeclaredFields() return? 3. How do annotations plus reflection let a framework validate any object? **Supplementary Problems –** • Add a @MaxLength(10) rule on a String field and test it. • Make the validator also print the field name with each error message. **Key Skills to be addressed –** Standard annotations (@Override, @FunctionalInterface); meta-annotations; defining custom annotations; reflection on fields and annotations. **Applications –** Validation frameworks, object-relational mappers and dependency-injection containers all read annotations by reflection. **Learning Outcome –** The student can define custom annotations and process them at run time with reflection. (CO1, CO2)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. An Account with an invalid (negative) balance is created in code.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-07. Confirm that an account with a negative balance produces the @Positive error message and a valid account produces none.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct custom annotations with meta-annotations – 35%; reflection-based validator – 35%; correct use of standard annotations – 10%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: an Account object whose balance is -100. **Process:** validate() reads each field, finds the @Positive annotation on balance, reads the value by reflection, and sees it is not greater than zero. **Expected Output:** The returned String[] contains one message, "must be > 0"; for a valid account the array is empty.	

Practical Number		CO/PO
8	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Guarded calculator: read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt. 2. Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0. Process a list of requests, catching and reporting each failure without stopping the run.	**CO3** PO3, PO4, PO5

Practical Number		CO/PO
	3. (For Advanced Learners – home) Create an AutoCloseable resource and use try-with-resources to prove it always closes — even when an exception is thrown inside the block — and that the original error is the one reported. Decide what to model as the resource. **Part B — Project Work (Hour 2): MiniBank** Until now a failed withdrawal just returned false. In this lab you make MiniBank fault-tolerant: you replace those flags with a custom exception hierarchy so every failure is explicit and carries information (such as the shortfall). This is the Milestone 1 checkpoint, where the model, validation, annotations and error handling come together. 1. In a package named exception, create a checked base class BankException that extends Exception. Then create InsufficientFundsException, AccountNotFoundException and InvalidAmountException, each extending BankException. Give InsufficientFundsException a long field shortfall. 2. Change withdraw(long amount) to declare throws InsufficientFundsException, InvalidAmountException; throw InsufficientFundsException when the balance is too low and InvalidAmountException when the amount is less than or equal to zero. Make deposit(long amount) throw InvalidAmountException for a non-positive amount. 3. Add a method transfer(Account to, long amount) that uses try, catch and finally and re-throws a BankException when needed. 4. In main, wrap the operations in try-catch blocks and print friendly messages; use a finally block; and show one try-with-resources example using a small AutoCloseable class that you create. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is the difference between a checked and an unchecked exception? 2. What is the purpose of the finally block, and when does it run? 3. How does try-with-resources guarantee a resource is closed? **Supplementary Problems –** • Add a DailyLimitExceededException and throw it when a withdrawal exceeds a daily cap. • Print the shortfall amount stored inside InsufficientFundsException. **Key Skills to be addressed –** try-catch-finally; checked vs unchecked exceptions; throw and throws; custom exception classes; exception propagation; try-with-resources. **Applications –** Robust error handling is mandatory in banking, payment and any system that must not crash on bad input. **Learning Outcome –** The student can build fault-tolerant code using custom exceptions and structured handling. (CO3)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. Operations that trigger each exception are called in code.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-08. Confirm that withdrawing more than the balance throws InsufficientFundsException and is caught with a clear message. Prepare for the Milestone 1 demo (classes + validation + annotations + exceptions).	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct exception hierarchy – 35%; throw/throws in deposit/withdraw/transfer – 30%; try-catch-finally and try-with-resources – 15%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: an account with balance 1000 on which withdraw(5000) is called.	

Practical Number		CO/PO
	Process: withdraw() finds the balance is insufficient and throws InsufficientFundsException with shortfall = 4000; the catch block in main handles it. **Expected Output:** The program does not crash; it prints a message such as "Withdrawal failed: short by 4000" and the balance stays 1000.	

Practical Number		CO/PO
9	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run. 2. Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do. 3. (For Advanced Learners – home) Split a large array of numbers across several threads that add into one shared total, show the wrong result, then fix it two ways and compare; decide how to measure the difference. **Part B — Project Work (Hour 2): MiniBank** A real bank serves many users at once. With the single-user logic working, this lab runs many threads against one account, lets you watch the balance come out wrong (a race condition), and then makes deposit and withdraw thread-safe with synchronization — protecting MiniBank's shared data. 1. Create a class named AccountWorker that implements Runnable with fields Account account, int times and long amount; in run(), loop times times and call account.deposit(amount). 2. In main, create one account, start about 10 threads using AccountWorker, call join() on each, then print the final balance — it will be wrong because of the race condition. 3. Make deposit and withdraw synchronized (or use a synchronized block on a private lock object) and run the program again — the balance is now correct. 4. Print the thread names to observe the thread lifecycle. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is a race condition, and why does it occur here? 2. What does the synchronized keyword do? 3. What are the states in a thread's life cycle? **Supplementary Problems –** • Replace synchronized with an AtomicLong balance and compare the results. • Add withdrawing threads as well and confirm the balance stays correct. **Key Skills to be addressed –** Thread and Runnable; starting and joining threads; the thread life cycle; race conditions; synchronized methods and blocks; thread safety. **Applications –** Any server handling many users at once — banking, ticketing, inventory — must make shared data thread-safe. **Learning Outcome –** The student can create threads and protect shared data using synchronization. (CO4)	**CO4** PO5, PSO1
	Dataset / Test Data (Source and Description, if applicable): No external dataset. The threads and their counts are set in code (e.g. 10 threads, 1000 deposits of 1 each).	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	

Practical Number		CO/PO
	Post Laboratory Work Description: Commit and tag lab-09. Record the wrong balance before synchronization and the correct balance after, and explain the difference.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Working multithreaded deposits – 30%; demonstrating the race condition – 25%; correct synchronization fix – 30%; viva – 15%.	
	Expected Input / Process / Output **Input:** In code: 10 threads, each calling deposit(1) one thousand times, on one account starting at balance 0. **Process:** Without synchronization the threads overwrite each other's updates; with synchronization each update completes before the next begins. **Expected Output:** Before the fix the printed balance is less than 10000 and varies each run; after the fix it is exactly 10000 every run.	

Practical Number		CO/PO
10	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused. 2. Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost. 3. (For Advanced Learners – home) Reproduce a deadlock between two threads that lock two objects in opposite order, then remove it with a consistent lock order. Choose the scenario yourself. **Part B — Project Work (Hour 2): MiniBank** Building on basic threads, this lab gives MiniBank a proper transaction engine. You process a batch of transactions on a managed thread pool, coordinate two threads with wait/notify, and learn to avoid deadlock with consistent lock ordering — the way back-office systems process many transactions safely. 1. Create a class named TransactionProcessor that holds an ExecutorService created with Executors.newFixedThreadPool(4). Add a method submit(Runnable task) that calls execute(), and a method stop() that calls shutdown() and awaitTermination(). 2. Create Runnable tasks that each perform a deposit or withdrawal; submit many of them, then stop the processor and wait for completion. 3. Build a simple producer-consumer: one thread adds transactions to a shared array buffer and another thread processes them, using wait() and notify() inside synchronized blocks. 4. Create a deadlock: two threads transfer money A to B and B to A, locking the two accounts in opposite order. Then fix it by always locking the account with the smaller accountNumber first. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. Why is a thread pool better than creating a new thread per task? 2. How do wait() and notify() coordinate two threads? 3. What causes a deadlock, and how does consistent lock ordering prevent it? **Supplementary Problems –** • Print how many tasks each pool thread executed. • Detect a long-running transfer and log a warning. **Key Skills to be addressed –** ExecutorService and thread pools; submitting Runnable tasks; inter-thread communication with wait/notify; deadlock and lock ordering. **Applications –** Batch processors, web-server request pools and job-scheduling systems all use managed thread pools.	**CO4** PO5, PO11, PSO1

Practical Number		CO/PO
	Learning Outcome – The student can run tasks on a thread pool, coordinate threads, and avoid deadlock. (CO4)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. A batch of transactions is created in code and submitted to the pool.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-10. Demonstrate the deadlock first, then show the fixed version completing normally.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Thread-pool processor – 30%; producer-consumer with wait/notify – 25%; deadlock created and fixed – 30%; viva – 15%.	
	Expected Input / Process / Output **Input:** In code: two threads, one transferring from account A to B and one from B to A, run with both lock orders. **Process:** With opposite lock orders each thread holds one account and waits for the other (deadlock); with the lower-number-first rule one thread completes before the other starts. **Expected Output:** The opposite-order version hangs (deadlock); the fixed version finishes and both balances are correct.	

Practical Number		CO/PO
11	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved. 2. Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes. 3. (For Advanced Learners – home) Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include. **Part B — Project Work (Hour 2): MiniBank** So far MiniBank forgets everything when it closes. In this lab you add the persistence and reporting layer: you make the model Serializable and save/load all accounts to a file, write an append-only transaction log, and then use NIO to walk the logs and produce an end-of-day summary report. 1. Make Account (and its subclasses and Customer) implement Serializable, and add a private static final long serialVersionUID field. 2. Create a class named StatePersister with a public static method void save(Account[] accounts, Path file) that uses ObjectOutputStream inside try-with-resources, and a public static method Account[] load(Path file) that uses ObjectInputStream. 3. Create a class named TransactionLog with a public static method void append(Path file, String line) that uses Files.write(...) with CREATE and APPEND. 4. Create a class named ReportGenerator that walks the logs folder using Files.newDirectoryStream, reads each file line by line with a BufferedReader, and adds	**CO5** PO4, PO5, PSO1, PSO2

Practical Number		CO/PO
	up the total deposits and total withdrawals in long variables; also read file details (size, last-modified) using Files.readAttributes, then write the summary to a report file. 5. In main, save the accounts, reload them into a new array and print them to confirm the data survived, then generate and print the report. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What does it mean for a class to be Serializable, and why add serialVersionUID? 2. What is the difference between a byte stream and a character stream? 3. How does NIO's Path/Files API differ from the old File class, and how is a directory stream iterated? **Supplementary Problems –** • Mark one field transient and show it is not saved. • Skip files that are empty or do not end with .log when generating the report. **Key Skills to be addressed –** Byte and character streams; object serialization and deserialization; Path and Files; try-with-resources; directory streams; file attributes; producing a report file. **Applications –** Saving application state, audit logging and end-of-day reporting are required in almost every program. **Learning Outcome –** The student can persist objects to files, reload them, and summarise log data using NIO. (CO5)	
	Dataset / Test Data (Source and Description, if applicable): The program creates a data folder at run time; the saved file (accounts.dat) and the log files are produced by the program itself and become the input to the report.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-11 (do not commit the generated data files). Confirm that data saved in one run is loaded correctly in the next, and that the report totals match the logged transactions.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Serializable model and save/load – 35%; transaction log – 15%; NIO report with correct totals – 30%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: an Account[] of three accounts saved to accounts.dat then reloaded; a logs folder with lines such as "DEPOSIT AC0001 500" and "WITHDRAW AC0002 200". **Process:** save() writes the array with ObjectOutputStream and load() reads it back; the report reads every log line and adds amounts to the deposit or withdrawal total based on the first word. **Expected Output:** The reloaded accounts print with the same numbers and balances as the saved ones; the report file shows the total deposits, total withdrawals and the net change.	

Practical Number		CO/PO
12	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Word frequency: split a paragraph into words, count each in a HashMap, and print the words sorted by frequency (highest first) using a Comparator.	**CO6** PO5, PSO1, PSO2

Practical Number		CO/PO
	2. Unique visitors: from a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names. 3. (For Advanced Learners – home) Build a leaderboard from player scores held in a Map and print the top-N sorted by score (ties broken by name); choose the data and sorting details. **Part B — Project Work (Hour 2): MiniBank** Until now accounts have lived in a plain array, which is hard to search and grow. In this lab you upgrade MiniBank's storage to the Collections Framework — a HashMap keyed by account number, sorting with a Comparator, and a thread-safe map for concurrent access. This is the Milestone 2 checkpoint. 1. Create a class named AccountStore that keeps accounts in a HashMap<String, Account> using the account number as the key. Add methods add(Account account), get(String accountNumber) and getAll(). 2. Keep a transaction history in an ArrayList<String>. 3. Add a method sortedByBalance() that returns an ArrayList<Account> sorted with a Comparator. 4. Make Account implement Comparable<Account> by comparing account numbers in compareTo(). 5. In the TransactionProcessor, use a ConcurrentHashMap for the shared accounts so it is thread-safe. 6. In main, add several accounts to the store and print them sorted by balance. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is the difference between a List, a Set and a Map? 2. When do you use a HashMap versus a TreeMap? 3. What is the difference between Comparable and Comparator? **Supplementary Problems –** • Add a method that returns all accounts with a balance above a given value. • Use a TreeMap so the accounts are kept in sorted key order. **Key Skills to be addressed –** List, Set and Map; HashMap and ArrayList; iterating; sorting with Comparable and Comparator; thread-safe collections. **Applications –** Almost every program stores and looks up data in collections; choosing the right one is a core engineering skill. **Learning Outcome –** The student can store, retrieve and sort objects using the Collections Framework. (CO6)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. Several accounts are added to the store in code.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-12. Confirm look-up by account number is correct and the sorted list is in the right order. Prepare for the Milestone 2 demo (persistence + collections).	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Map-based store and ArrayList history – 35%; sorting with Comparator/Comparable – 25%; thread-safe collection – 15%; naming conventions and viva – 25%.	
	Expected Input / Process / Output **Input:** In code: three accounts with balances 5000, 2000 and 9000 added to the store.	

Practical Number		CO/PO
	Process: get() looks an account up by its number in the HashMap; sortedByBalance() copies the values and sorts them with a Comparator. **Expected Output:** get("AC0002") returns the correct account; the sorted list prints the accounts in the order 9000, 5000, 2000.	

Practical Number		CO/PO
13	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Generic containers: a generic Box<T> with get/set and a Pair<A,B>; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array. 2. Bounded stats: a generic method <N extends Number> double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses non-Number types. 3. (For Advanced Learners – home) Implement a small generic container of your choice (e.g. a stack) backed by an array, reuse it for two different types, and handle the empty case sensibly. **Part B — Project Work (Hour 2): MiniBank** The HashMap store from the previous lab works only for accounts. In this lab you make MiniBank's storage reusable: one generic Repository<T> that stores any type safely and finds each object's key through the @Id annotation by reflection — the repository pattern used in real data-access layers. The service layer in Practical 14 will use it. 1. Create a generic class named Repository<T> that holds a HashMap<String, T>. Add methods T save(T entity), T findById(String id), ArrayList<T> findAll() and boolean delete(String id). 2. Inside save(), find the id by reflection: look for the field marked @Id (from Practical 7) and read its value as the map key. 3. Add a method marked @SafeVarargs called saveAll(T... items) that saves several objects at once. 4. Add one generic method with a bounded type parameter, for example static <N extends Number> double average(N[] values). 5. In main, create a Repository<Account>, save several accounts, fetch one by id, and print all of them. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What problem do generics solve compared with using Object? 2. What is a bounded type parameter, and why is <N extends Number> useful? 3. What is type erasure? **Supplementary Problems –** • Create a Repository<Customer> and reuse the same class without changes. • Add a method that returns the number of stored items. **Key Skills to be addressed –** Generic classes and methods; bounded type parameters; wildcards; type erasure; reflection to read the @Id field. **Applications –** Generic repositories and collections (like those in the JDK and Spring Data) let one class serve many types safely. **Learning Outcome –** The student can write reusable, type-safe components using generics. (CO6)	**CO6** PO4, PO5, PSO1, PSO2
	Dataset / Test Data (Source and Description, if applicable): No external dataset. Account (and optionally Customer) objects are saved in code.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work.	

Practical Number		CO/PO
	• **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-13. Confirm the same Repository class works for more than one type and that findById returns the correct object.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Generic Repository with CRUD – 40%; reflection on @Id – 20%; bounded type / varargs method – 20%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: three Account objects saved into a Repository<Account>, then findById called with one account number. **Process:** save() reads each account's @Id field by reflection and stores it under that key; findById() looks the key up in the map. **Expected Output:** findById returns the matching account; findAll() prints all three; the same Repository class also works for Customer objects.	

Practical Number		CO/PO
14	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank. 2. (For Advanced Learners – home) Organise that app into packages, build a runnable JAR, and add a test that saves and reloads its data unchanged. **Part B — Project Work (Hour 2): MiniBank** All the parts now exist. In this final capstone lab you assemble MiniBank and finish it to a deliverable standard: a BankService wires the repository, validation and exceptions together and the Practical 1 menu calls real operations; you add safe batch processing and the report; and you test and package the application. This is the Milestone 3 final checkpoint and viva. 1. Create a class named BankService that uses Repository<Account>, the Validator and the exception classes to provide openAccount, deposit, withdraw, transfer, statement and listAccounts. 2. Connect the Practical 1 menu so each option calls the matching BankService method; load saved accounts on start-up (Practical 11) and save them on exit. 3. Use the TransactionProcessor (Practical 10) over the thread-safe store (Practical 12) to process a batch of transactions safely, and add the report (Practical 11) as a menu option. 4. Write a few JUnit tests using @Test: a deposit increases the balance; a withdrawal over the limit throws an exception; many concurrent deposits keep the balance correct. 5. (Optional) Add an @Audit annotation and a reflective logger that records which service method was called. 6. Build the final runnable JAR and complete the README. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is a layered architecture (model, service, user interface), and why is it useful? 2. How do you assert in JUnit that a method throws an expected exception? 3. What does packaging the application as a runnable JAR provide? **Supplementary Problems –** • Add a "search account by owner name" option. • Add a test that saves and loads state and checks the data is unchanged. **Key Skills to be addressed –** Layered design and module integration; wiring user interface to service to repository; concurrency integration; unit testing with JUnit and @Test; building and packaging a runnable JAR; documentation.	**CO1–CO6** PO3, PO5, PO11, PSO1, PSO2

Practical Number		CO/PO
	Applications – Every real application is assembled, tested and packaged in exactly this way; this is everyday professional software delivery. **Learning Outcome –** The student can integrate all modules into one tested, packaged, concurrent application — industry-ready. (CO1–CO6)	
	Dataset / Test Data (Source and Description, if applicable): The accounts file saved by Practical 11 loads at start-up; the user drives the program through the menu; test data is created inside the JUnit tests.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), JUnit 5, command line (javac, jar, java), git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 3 hours (capstone).	
	Post Laboratory Work Description: Commit and tag lab-14 (final). Ensure all tests pass, the JAR runs, the README explains how to build and run, and data persists across restarts. Prepare for the Milestone 3 final demo and individual viva.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Working BankService, menu wiring and persistence – 35%; concurrent batch over the thread-safe store – 15%; passing JUnit tests – 25%; runnable JAR, README, final demo and viva – 25%.	
	Expected Input / Process / Output **Input:** From the menu: open an account, deposit, withdraw, transfer, view the statement, run the report; the JUnit tests run by the test runner. **Process:** Each menu choice calls a BankService method that validates input, updates the Repository and reports success or a caught exception; the processor runs the batch on the ConcurrentHashMap store; the tests assert the rules. **Expected Output:** The statement reflects every operation; data persists across restarts; all JUnit tests pass; java -jar runs the finished MiniBank.	